

## Overview: Object-Oriented Development

Before diving into design, it is important to understand where it fits in. According to Sommerville and your Unit 3 slides, Object-Oriented Development comprises three related but distinct stages:

1. **Object-Oriented Analysis (OOA):** Developing an object model of the *application domain* (real-world entities).
2. **Object-Oriented Design (OOD):** Developing an object-oriented system model to *implement requirements* (the software solution).
3. **Object-Oriented Programming (OOP):** Realizing the design using an OO programming language like Java or C++.

**Characteristics of Object-Oriented Design (OOD):** In OOD, a system is designed as a set of interacting objects that manage their own state and operations.

- **Encapsulation:** Objects are independent entities that encapsulate (hide) their state and representation information.
- **No Shared Data:** Shared data areas are eliminated. Objects communicate exclusively by message passing.
- **Real-world Abstraction:** Objects are abstractions of real-world or system entities.
- **Distribution:** Objects may be distributed and can execute sequentially or in parallel.

### Advantages of OOD:

- **Easier Maintenance:** Because objects are standalone entities, changing the internal implementation of one object does not break others.
- **Reusability:** Objects act as independent, reusable components.
- **Understandability:** There is often a clear mapping from real-world entities to system objects.

---

## TOPIC 1: Objects and Object Classes

### 1. Core Definitions

- **Object:** An entity that has a private state and a defined set of operations that operate on that state. The state is represented as a set of object attributes. The operations provide services to other objects.
- **Object Class:** A blueprint or template used to create objects. It declares all the attributes and operations that an object of that specific class will possess.

#### Diagram Directions: Object Classes (UML Notation)

- **Symbol Definition:** A vertically oriented rectangle divided into three horizontal sections.
- **Directions:**
  1. **Top section:** Write the Name of the class (e.g., **Employee**).
  2. **Middle section:** List the Attributes/State variables (e.g., **name: string, address: string**).
  3. **Bottom section:** List the Operations/Methods (e.g., **join(), leave()**).

### 2. Object Communication (Message Passing)

Since there is no shared data, objects communicate via **message passing**. A message consists of:

1. The name of the service/method requested by the calling object.
2. Copies of the information/parameters required to execute the service. *Implementation:* In practice, this is implemented as a standard procedure or method call (e.g., **thermostat.setTemp(20);**).

### 3. Generalisation and Inheritance

Classes can be arranged into hierarchies. A **super-class** represents a general concept, while **sub-classes** represent specialized versions.

- **How it works:** A sub-class inherits the attributes and operations from its super-class but can add its own new attributes or methods.
- **Advantages:** It is an excellent abstraction and reuse mechanism.
- **Problems with Inheritance:** Object classes are not self-contained—you cannot understand a sub-class without looking at its super-classes. Also, designers often mistakenly reuse analysis graphs for implementation, leading to inefficiencies.

## Diagram Directions: Generalisation Hierarchy

- **Symbol Definition:** Solid lines connecting class rectangles, ending in a **hollow triangle** (arrowhead).
- **Directions:** Draw the super-class at the top (e.g., **Employee**). Draw sub-classes below it (e.g., **Manager**, **Programmer**). Draw a line from the sub-classes pointing *upwards* to the super-class, with the hollow triangle touching the super-class.

## 4. UML Associations & Aggregation

Objects participate in relationships with each other. In UML, a general relationship is called an **association**.

- **General Association:** A solid line connecting two object classes, annotated with text describing the relationship (e.g., **Employee is-member-of Department**).
- **Aggregation:** A specific association showing that an object is composed of (an aggregate of) other objects (e.g., a **Study Pack** consists of **Lecture Notes** and **Exercises**).
  - **Diagram Directions:** Draw a solid line with a **diamond shape** placed on the end of the line *touching the "Whole/Container" object*.

## 5. Concurrent Objects

Because objects are self-contained entities, they are naturally suited for concurrent (parallel) implementation.

1. **Servers (Passive objects):** These objects execute as parallel processes but suspend themselves while waiting for a message. They only act when triggered by an external request.
2. **Active objects:** These objects change their own state autonomously without outside intervention. They typically contain an infinite loop so they are continuously computing or polling (e.g., a transponder continuously updating its position).

**PYQ Exemplar 1:** *"Differentiate between the two kinds of concurrent objects."* (Aug 2024)

- **What to do:** Create a clear comparison. Define **Passive Objects (Servers)** as entities that wait for an external message trigger to act, and **Active Objects** as autonomous entities containing an infinite loop that continuously update their own state.
- **How to score most:** Use the keywords "suspends itself," "external trigger," "autonomous," and "infinite loop."

**PYQ Exemplar 2:** *"Design an object aggregation model for a ticket reservation system..."* (Feb 2025) OR *"Draw object aggregate diagram for a study pack..."* (June 2025)

- **What to do:** Draw the main container box (e.g., "Ticket System" or "Study Pack"). Connect it to component boxes ("Tickets", "Customers" or "Lecture Notes").
- **What NOT to do:** Do not use standard arrowheads. You **must** draw the diamond symbol, and it **must** be touching the main container object to get the marks for aggregation.

**PYQ Exemplar 3:** *"Identify the objects and design object Generalization diagram showing interactions of objects for a group diary and time management system..."* (June 2025)

- **What to do:** Identify a parent class (e.g., **User**) and sub-classes (e.g., **Manager**, **Employee**). Connect them with a solid line ending in a **hollow triangle** pointing toward the parent class.

---

## TOPIC 2: An Object-Oriented Design Process

There is no single "magic formula" for OOD. It is a highly iterative, creative process. Sommerville and Unit 3 define a framework involving **five interleaved stages**:

### Stage 1: System Context and Models of Use

Before designing, you must understand how the software relates to its external environment.

- **System Context (Static Model):** Describes the other external systems that surround your system. Usually shown using UML Packages.
- **Models of Use (Dynamic Model):** Describes how users/actors interact with the system, represented using **Use-Case diagrams**.

### Stage 2: Architectural Design

Once you understand the interactions, you establish a high-level structural framework by breaking the system down into major sub-systems.

- **Example:** A weather station system is designed using a **Layered Architecture** (e.g., Interface layer, Data Collection layer, Instruments layer).

- **Diagram Directions:** Use UML Packages (rectangles with a small tab on the top-left, like a folder). Label them with «**subsystem**».

### Stage 3: Object Identification

Identifying objects (or object classes) is the most difficult part of OOD. Analysts use four major approaches to discover objects:

1. **Grammatical Approach:** Analyzing a natural language description of the problem. *Nouns* become objects/attributes; *verbs* become operations.
2. **Tangible Entities:** Identifying physical, real-world things (e.g., Anemometer, Thermometer, Car, Manager).
3. **Behavioral Approach:** Understanding the overall behavior of the system and identifying participants who initiate those behaviors.
4. **Scenario-based Analysis:** Analyzing use-cases step-by-step to identify the required objects, attributes, and methods as the scenario plays out.

### Stage 4: Design Models

Design models bridge the requirements and the code, showing objects and their relationships.

- **Static Models:** Describe the static structure of the system in terms of object classes and relationships (e.g., Class models, Sub-system models).
- **Dynamic Models:** Describe the dynamic interactions between objects.
  - **Sequence Models:** Show the sequence of object interactions over time.
    - **Directions:** 1) Arrange participating objects horizontally across the top. 2) Draw a dashed vertical line (lifeline) hanging down from each object representing time flowing downwards. 3) Draw labeled horizontal arrows between lifelines to represent messages. 4) **Critical Symbol:** Draw a *thin vertical rectangle* (*activation box*) over the dashed line to show when the object is actively controlling the system.
  - **State Machine Models:** Show how an individual object changes state in response to events.
    - **Directions:** Draw rounded rectangles for states (e.g., **Waiting, Testing**). Draw directed arrows between them labeled with the event/trigger (e.g., **calibrate()**) that forces the transition.

### Stage 5: Object Interface Specification

Object interfaces have to be specified so that objects and components can be designed in parallel by different teams.

- **Rule:** Hide the internal data representation within the object. Only expose the methods/operations.
- Interfaces can be defined in UML, but an alternative (often preferred) approach is to use a programming language like **Java** (using **interface { ... }** blocks) because the compiler can catch syntax errors early.

**PYQ Exemplar 1:** "Analyze the various proposals that help in identifying objects." (June 2025)

- **What to do:** List and briefly explain all 4 approaches (Grammatical, Tangible Entities, Behavioral, Scenario-based).
- **How to score most:** Provide a one-line example for each to prove clarity (e.g., "In the grammatical approach, a noun like 'Customer' becomes an object, while the verb 'buy' becomes an operation").

**PYQ Exemplar 2:** "Using the UML graphical notation... design Sequence diagram for a petrol or gas station to be set up for fully automated operation." (Aug 2024, June 2025)

- **What to do:** Set up objects horizontally (e.g., **Customer, Pump Controller, Payment Gateway**). Draw the dashed lifelines downward. Draw horizontal arrows for messages like **swipeCard()** or **dispenseFuel()**.
- **What NOT to do:** Do not draw arrows diagonally. Time flows downwards, but message arrows must be strictly horizontal. **Do not forget the thin vertical activation rectangles** on the dashed lifelines—this is the most common place students lose marks.

---

### TOPIC 3: Design Evolution

After a system is deployed, it is inevitable that new requirements will emerge and the system will need to evolve (change).

**How OOD Simplifies Evolution:** Object-oriented design is exceptionally well-suited to system evolution because it strictly adheres to **Information Hiding (Encapsulation)** and **Loose Coupling**.

- Because an object's internal state is private, changing the internal algorithms or data structures of one object will not ripple out and unpredictably break other objects.
- Because objects are loosely coupled and interact only through standardized public interfaces, you can plug entirely new objects into the system seamlessly.

**The Weather Station Example (From Sommerville & Unit 3):** Assume an automated weather station needs to be upgraded to monitor pollution (e.g., nitrous oxide, smoke).

- In an OOD system, you simply create a new **Air quality** object class and related hardware objects (**NOmeter**, **SmokeMeter**).
- You add an operation **reportAirQuality()** to the main **WeatherStation** object.
- The original **Ground thermometer** or **Anemometer** objects remain completely untouched and unaware of the change. The addition of pollution data does not affect weather data collection in any way, drastically reducing maintenance costs and bugs.

*Exam Trend Note:* While not explicitly a 10-mark standalone question in the provided PYQs, this concept is highly favored for 4 to 5-mark justification questions in Part B.

**Expected Question:** *"Justify how Object-Oriented Design supports design evolution and makes software easier to maintain."*

• **What to do:** Structure your answer explicitly around the two keywords: **Information Hiding** and **Loose Coupling**.

• **How to score most:** Do not write generic statements like "OOD is easy." Use the specific "Weather Station adding a Pollution Monitor" example from the syllabus to practically prove your point. State clearly that because of encapsulation, the original sensors required zero code rewriting.